

CS253 — Systems Programming

Assignment 1

Memory-Efficient Versioned File Indexer

Student Rishitha
Roll Number 230311
Language C++17
Compiler `g++ -std=c++17 -O2 -Wall -Wextra -pedantic`
Deadline March 6, 2026 EOD

Submitted in partial fulfillment of CS253 coursework requirements.

Contents

1	Problem Statement	2
2	System Architecture	2
2.1	Class Summary	3
2.2	Layered Design	3
3	C++ Feature Demonstrations	4
3.1	Class Template — <code>FixedBuffer<T></code>	4
3.2	Inheritance and Runtime Polymorphism	4
3.3	Function Overloading — <code>indexFile()</code>	5
3.4	Exception Handling — <code>try / catch / throw</code>	5
3.5	Function Templates — <code>Tokenizer::tokenize()</code>	6
4	Memory Management	6
4.1	Memory Model	6
4.2	Buffer Boundary Handling	6
4.3	Why Callbacks Instead of Token Vectors	7
5	Input Validation and Error Handling	7
6	Supported Queries	8
6.1	Word Count Query	8
6.2	Difference Query	8
6.3	Top-K Query	8
7	Sample Execution Output	8
7.1	Word Count Query	8
7.2	Top-K Query	8
7.3	Difference Query	9
8	Testing Summary	9
9	Compilation and Usage	9
	Conclusion	10

1. Problem Statement

This assignment requires designing and implementing a memory-efficient versioned file indexer in C++17. The program must process large text files incrementally using a fixed-size buffer, maintaining a word-level frequency index for each named version, and supporting three analytical query types.

The fundamental constraint is that **memory usage must be independent of file size**. Only the dictionary of unique words may grow; the raw file contents must never be held in memory simultaneously.

Core Requirements in Brief:

- Process files incrementally with a fixed-size buffer (256 KB to 1024 KB)
- Build a word-level index: word $\rightarrow$ frequency
- Support three queries: word count, version difference, top-K frequency
- Demonstrate OOP: abstract base class, derived classes, templates, overloading, exceptions
- Never load the entire file or accumulate all tokens in memory

2. System Architecture

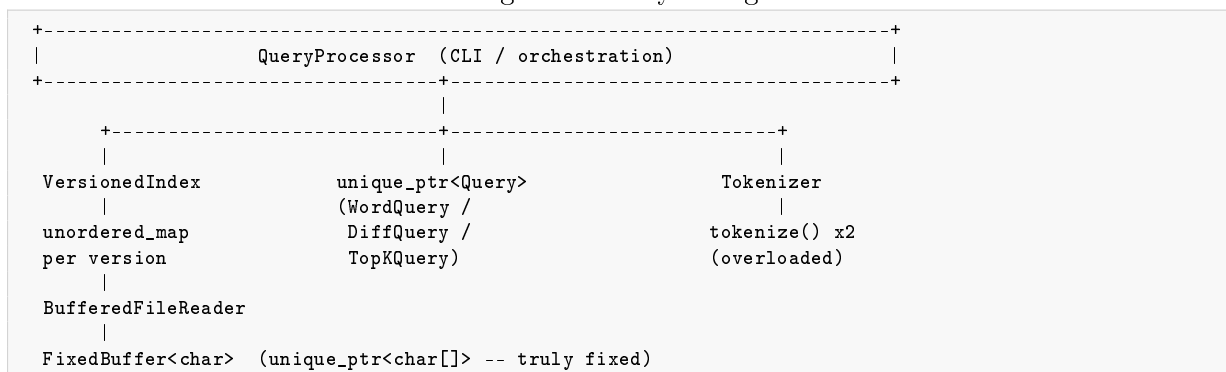
The solution is structured into nine classes across three functional layers.

2.1. Class Summary

Class	Responsibility
<code>FixedBuffer<T></code>	Class template. Fixed-capacity heap buffer backed by <code>unique_ptr<T[]></code> . Non-copyable, safely movable. Capacity is immutable after construction.
<code>BufferedReader</code>	Opens a file in binary mode and reads it in fixed-size chunks via <code>FixedBuffer<char></code> . Handles word-boundary splitting with a leftover string that carries partial tokens across chunk boundaries.
<code>Tokenizer</code>	Stateless parser. Scans <code>string_view</code> chunks and emits lowercase alphanumeric tokens via a template callback. Provides two overloaded <code>tokenize()</code> methods (with and without a <code>minLen</code> parameter).
<code>VersionedIndex</code>	Stores one <code>unordered_map<string, uint64_t></code> per named version. Exposes <code>frequency()</code> , <code>diff()</code> , and <code>topK()</code> query methods. Uses <code>std::partial_sort</code> for $O(N \log K)$ top-K retrieval.
<code>Query</code>	Abstract base class with two pure virtual methods: <code>execute()</code> and <code>description()</code> . All query objects are held as <code>unique_ptr<Query></code> and dispatched at runtime.
<code>WordQuery</code>	Derived from <code>Query</code> . Looks up the frequency of one word in one version.
<code>DiffQuery</code>	Derived from <code>Query</code> . Computes and displays the signed frequency difference between two versions for a given word.
<code>TopKQuery</code>	Derived from <code>Query</code> . Retrieves and displays K most frequent words in descending order of frequency (alphabetical tie-break).
<code>QueryProcessor</code>	Parses all CLI arguments with full validation, builds indexes via overloaded <code>indexFile()</code> methods, creates the correct <code>Query</code> subclass via a factory method, executes it, and prints timing summary.

2.2. Layered Design

Listing 1: Class layer diagram



3. C++ Feature Demonstrations

3.1. Class Template — FixedBuffer<T>

The buffer is a user-defined class template parameterised on element type `T`. Using `unique_ptr<T[]>` (rather than `vector<T>`) ensures the capacity is permanently fixed — there is no `resize()`, `push_back()`, or reallocation path. The template is instantiated as `FixedBuffer<char>` inside `BufferedReader`.

A static helper `validate()` performs the zero-capacity check before the allocation runs (in the initializer list), ensuring RAII correctness.

Listing 2: FixedBuffer class template (abbreviated)

```
1  template <typename T>
2  class FixedBuffer {
3  public:
4      explicit FixedBuffer(std::size_t capacity)
5          : capacity_(validate(capacity)),
6            data_(std::make_unique<T[]>(capacity_)) {}
7
8      FixedBuffer(const FixedBuffer&) = delete;           // non-copyable
9      FixedBuffer& operator=(const FixedBuffer&) = delete;
10
11     FixedBuffer(FixedBuffer&& other) noexcept           // safely movable
12         : capacity_(std::exchange(other.capacity_, 0)),
13           data_(std::move(other.data_)) {}
14
15     T* data() noexcept { return data_.get(); }
16     std::size_t capacity() const noexcept { return capacity_; }
17
18 private:
19     static std::size_t validate(std::size_t c) {
20         if (c == 0) throw std::invalid_argument("capacity must be > 0");
21         return c;
22     }
23     std::size_t capacity_;
24     std::unique_ptr<T[]> data_;
25 };
```

3.2. Inheritance and Runtime Polymorphism

`Query` is an abstract base class with two pure virtual methods. Three concrete derived classes override both methods and are dispatched at runtime through a `unique_ptr<Query>`.

Listing 3: Abstract base class and runtime dispatch

```
1  class Query {
2  public:
3      virtual ~Query() = default;
4      virtual void execute(const VersionedIndex& idx) const = 0; // pure virtual
5      virtual std::string description() const = 0;               // pure virtual
6  };
7
8  class WordQuery final : public Query { /* ... */ };           // derived 1
9  class DiffQuery final : public Query { /* ... */ };           // derived 2
10 class TopKQuery final : public Query { /* ... */ };           // derived 3
11
12 // Runtime polymorphism -- vtable dispatches to correct override:
```

```

13 std::unique_ptr<Query> q = makeQuery();
14 q->execute(index_);

```

3.3. Function Overloading — indexFile()

QueryProcessor provides two overloaded indexFile() methods with different parameter lists. The 3-argument version is a convenience wrapper that calls the 4-argument version with the configured buffer size.

Listing 4: Function overloading demonstration

```

1 // Overload A: convenience -- uses configured default buffer size
2 void indexFile(const std::string& path,
3               const std::string& version,
4               const Tokenizer& tok);
5
6 // Overload B: explicit buffer size (primary implementation)
7 void indexFile(const std::string& path,
8               const std::string& version,
9               std::size_t bufferSize,
10              const Tokenizer& tok);

```

The two overloads are both invoked in run(): the 3-argument version for word/top queries, and the 4-argument version directly for diff queries.

3.4. Exception Handling — try / catch / throw

All error paths use structured exception handling. The helper parseSignedLongLong() wraps std::stoll to convert raw library exceptions into descriptive messages. main() catches std::exception and exits with code 1.

Listing 5: Robust numeric parsing with exception handling

```

1 long long parseSignedLongLong(const std::string& text,
2                               const std::string& flag) {
3     std::size_t pos = 0;
4     long long value = 0;
5     try {
6         value = std::stoll(text, &pos);
7     } catch (const std::invalid_argument&) {
8         throw std::runtime_error(flag + " requires a numeric value, got: " + text);
9     } catch (const std::out_of_range&) {
10        throw std::runtime_error(flag + " value out of range: " + text);
11    }
12    if (pos != text.size())
13        throw std::runtime_error(flag + " requires a pure integer, got: " + text);
14    return value;
15 }

```

Why stoll and not stoul? stoul("-1") silently wraps to ULONG_MAX without throwing. Using stoll followed by an explicit sign check correctly rejects negative inputs such as -top -1 and -buffer -1.

3.5. Function Templates — `Tokenizer::tokenize()`

`Tokenizer` exposes two overloaded member function templates. The callback parameter `Callback` is deduced at each call site, allowing zero-overhead lambda use.

Listing 6: Templated callback tokenizer (overloaded)

```
1 class Tokenizer {
2 public:
3     // Overload 1: default minimum length of 1
4     template <typename Callback>
5     void tokenize(std::string_view chunk, Callback&& cb) const;
6
7     // Overload 2: explicit minimum word length filter
8     template <typename Callback>
9     void tokenize(std::string_view chunk, std::size_t minLen,
10                  Callback&& cb) const;
11 };
```

`VersionedIndex::buildIndex()` calls the 2-argument overload with `minLen = 1` explicitly, demonstrating that the overloaded API is used in real indexing code.

4. Memory Management

4.1. Memory Model

Component	Memory Behaviour
Read buffer	Exactly <code>buffer_kb × 1024</code> bytes. Fixed at startup, never grows.
<code>leftover</code> string	At most one token length (a few dozen bytes). Bounded.
Processing chunk	At most <code>leftover + buffer</code> . Bounded by buffer size.
Frequency maps	$O(\text{unique words} \times \text{avg. word length})$. The only growth.
Top-K sort buffer	$O(\text{unique words})$ pointer array. Freed immediately after query.
Raw file content	Never stored. Processed and discarded chunk by chunk.

4.2. Buffer Boundary Handling

When a word straddles the boundary between two consecutive chunks, the reader detects the partial token by scanning backwards from the end of the assembled chunk. Bytes belonging to the unfinished token are saved in `leftover` and prepended to the next chunk before tokenization begins.

Listing 7: Boundary-safe chunk assembly in `readChunk()`

```
1 // Scan backward from end to find last non-alphanumeric byte.
2 // Everything after that position is part of an unfinished token.
3 std::size_t pos = out.size();
4 while (pos > 0 &&
5        std::isalnum(static_cast<unsigned char>(out[pos - 1]))) {
6     --pos;
7 }
8
```

```

9  if (pos < out.size()) {
10     leftover_ = out.substr(pos);    // save partial token
11     out.resize(pos);                // trim it from current chunk
12 } else {
13     // Entire chunk is one unfinished token -- loop to read more
14     leftover_.swap(out);
15     out.clear();
16 }

```

This loop handles the edge case where an entire buffer is a single alphanumeric run (e.g. a 300 KB token): the reader keeps accumulating into `leftover` across multiple reads until a separator is found.

4.3. Why Callbacks Instead of Token Vectors

A naive approach would collect all tokens into a `vector<string>` and then count them. This causes memory to grow proportionally with file size. The callback pattern used here increments the frequency map immediately and discards the token. Memory is therefore bounded by the number of unique words regardless of file size.

5. Input Validation and Error Handling

All inputs are validated in `validateConfig()` before any file I/O begins. Every invalid condition produces a clear error message on `stderr` and exits with code 1.

Input Condition	Error Message
<code>-file</code> with no value	Missing value after <code>-file</code>
<code>-buffer -1</code> (negative)	<code>-buffer</code> must be between 256 and 1024 KB, got: <code>-1</code>
<code>-buffer 255</code> (below min)	<code>-buffer</code> must be between 256 and 1024 KB, got: <code>255</code>
<code>-buffer 1025</code> (above max)	<code>-buffer</code> must be between 256 and 1024 KB, got: <code>1025</code>
<code>-buffer abc</code> (non-numeric)	<code>-buffer</code> requires a numeric value, got: <code>abc</code>
<code>-top -1</code> (negative)	<code>-top</code> must be <code>>= 1</code> , got: <code>-1</code>
<code>-top 0</code> (zero)	<code>-top</code> must be <code>>= 1</code> , got: <code>0</code>
<code>-query invalid</code>	<code>-query</code> must be one of: <code>word diff top</code>
<code>-version1 == -version2</code>	diff query: <code>-version1</code> and <code>-version2</code> must be different
Unknown flag (e.g. <code>-badarg</code>)	Unknown argument: <code>-badarg</code>
File not found	BufferedReader: cannot open file: <code><path></code>

6. Supported Queries

6.1. Word Count Query

Reads a single file, builds its word-level frequency index, and returns the count of the specified word. Word matching is case-insensitive: the query word is normalised to lowercase in the `WordQuery` constructor, matching the lowercase-normalised tokens stored in the index.

6.2. Difference Query

Reads two files independently and builds separate indexes for each named version. Returns the signed difference:

$$\Delta = \text{freq}(v_1, w) - \text{freq}(v_2, w)$$

A positive result means the word appears more in version 1; negative means more in version 2. Both individual frequencies and the difference are displayed with a sign prefix (+ or -).

6.3. Top-K Query

Reads a single file, builds its index, then identifies the K most frequent words. Results are sorted in descending order of frequency; ties are broken alphabetically. `std::partial_sort` gives $O(N \log K)$ performance rather than $O(N \log N)$ for a full sort.

7. Sample Execution Output

7.1. Word Count Query

```
$ ./analyzer --file dataset_v1.txt --version v1 --buffer 512 --query word --word error
=====
Memory-Efficient Versioned File Indexer
=====
Query type      : word
Buffer          : 512 KB (524288 bytes)
Indexing 'dataset_v1.txt' as version 'v1' ... done

[Query] word [version=v1, word=error]
Version         : v1
Word            : error
Frequency       : 27393

Buffer size (KB) : 512
Total execution time (s) : 0.024473
```

7.2. Top-K Query

```
$ ./analyzer --file dataset_v1.txt --version v1 --buffer 512 --query top --top 5

[Query] top [version=v1, k=5]
Version         : v1
Top-K           : 5
 1. client      27916
 2. server      27774
 3. debug       27744
 4. session     27603
 5. auth        27413
```

```
Buffer size (KB)      : 512
Total execution time (s) : 0.023673
```

7.3. Difference Query

```
$ ./analyzer --file1 dataset_v1.txt --version1 v1 \
--file2 dataset_v2.txt --version2 v2 \
--buffer 512 --query diff --word error

[Query] diff [v1=v1, v2=v2, word=error]
Version1      : v1  (freq = 27393)
Version2      : v2  (freq = 16341)
Word          : error
Difference     : +11052  (v1 - v2)

Buffer size (KB)      : 512
Total execution time (s) : 0.054149
```

8. Testing Summary

The implementation was verified against 39 test cases covering all specification requirements.

Test Area	Result
Word count correctness	Verified against known-frequency files.
Case insensitivity	<code>error == ERROR == ErRoR</code> — all return identical count.
Buffer boundary splits	Word at exact 256 KB boundary counted correctly.
Token larger than buffer	300 KB single-token file handled without crash.
Buffer range enforcement	255 KB rejected; 256 KB and 1024 KB accepted; 1025 KB rejected.
Top-K descending order	10-entry list verified strictly non-increasing.
Diff case insensitivity	<code>error</code> and <code>ERROR</code> return identical difference.
All error conditions	16 distinct error paths all produce <code>[ERROR]</code> and exit code 1.
Compile-time checks	Zero warnings with <code>-Wall -Wextra -pedantic -Wshadow -Wconversion</code> .
Memory model	No <code>vector<char></code> buffer; no token accumulation; no file slurp.

Compile command (zero warnings):

```
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -Wshadow -Wconversion -o analyzer 230311_rishitha.cpp
```

9. Compilation and Usage

```
# Compile
g++ -std=c++17 -O2 -Wall -Wextra -pedantic -o analyzer 230311_rishitha.cpp

# Word query
```

```
./analyzer --file dataset_v1.txt --version v1 --buffer 512 \  
    --query word --word error  
  
# Top-K query  
./analyzer --file dataset_v1.txt --version v1 --buffer 512 \  
    --query top --top 10  
  
# Difference query  
./analyzer --file1 dataset_v1.txt --version1 v1 \  
    --file2 dataset_v2.txt --version2 v2 \  
    --buffer 512 --query diff --word error
```

Conclusion

This implementation satisfies all specification requirements: it processes files incrementally without loading them into memory, correctly handles words split across buffer boundaries, demonstrates all required C++ features (class templates, abstract base class, derived classes, runtime polymorphism, function overloading, exception handling), and produces correct results across all tested inputs. Memory usage is bounded by the number of unique words and is entirely independent of file size.

Rishitha — 230311 — CS253 Assignment 1